

Serviço Nacional de Aprendizagem Comercial - SENAC

Engenharia da Computação

Automatização de testes de segurança com IHM

Grupo 11 IHM

Álvaro Pereira Araújo

Lucas Fernandes Miranda

Lukas de Andrade Xavier

Guilherme Inacio

Leonardo Nascimento

São Paulo
Junho de 2025

Resumo

Este artigo contempla as etapas de pesquisa, desenvolvimento, prototipagem e integração web de um sistema de automação voltado para monitoramento de pressão em sistemas de ar ou fluidos, utilizando microcontroladores de baixo custo. O projeto foi desenvolvido por alunos do 5º semestre do curso de Engenharia com o objetivo de aplicar conceitos de instrumentação, automação e Internet das Coisas (IoT). A solução utiliza o ESP32 conectado a um sensor de pressão MPX5700 para leitura de dados, os quais são exibidos em tempo real em um display LCD 1602A e também enviados para uma aplicação web via Wi-Fi. O sistema demonstra a viabilidade de leitura analógica precisa, exibição local e integração com interface web, podendo ser expandido para aplicações clínicas, industriais ou laboratoriais.

Palavras-chave: Engenharia da Computação, automação, IHM, ESP32, sensor de pressão.

Sumário

1	Introdução	3
2	Objetivos	3
2.1	Objetivo geral	3
2.2	Objetivos específicos	3
3	Metodologia	3
4	Desenvolvimento do Sistema	4
4.1	Arquitetura do sistema elétrico	4
4.2	Programação embarcada (ESP32)	4
4.3	Programação do servidor web (PHP)	7
4.4	Desenvolvimento do site (visualização gráfica responsiva)	7
4.5	Orçamento dos componentes	12
4.6	Testes de funcionamento e resultados	12
5	Discussões e Limitações	13
6	Conclusão	13
7	Referências	13

1. Introdução

Este projeto foi desenvolvido durante o 5º semestre do curso de Engenharia da Computação (BEC), integrando as atividades da disciplina de Projeto: Simulação com Modelagem, Operação e Gestão I. O trabalho foi realizado com a parceria da empresa Oppler, que atua na área de automação e forneceu uma de suas placas baseadas no ESP32.

O objetivo foi explorar aplicações de leitura de sensores analógicos, exibição de dados locais e integração futura com sistemas remotos, aplicando conceitos de automação, IoT e IHM (Interface Homem-Máquina). O microcontrolador ESP32 foi utilizado em conjunto com o sensor de pressão MPX5700, e os dados foram exibidos em um display LCD 1602A e enviados para uma interface web de monitoramento em tempo real.

Durante o desenvolvimento, o grupo aproveitou experiências de projetos anteriores, o que facilitou a integração entre hardware e software, especialmente quanto à organização dos componentes e lógica embarcada.

2. Objetivos

2.1 Objetivo geral

Promover o desenvolvimento de um sistema embarcado de monitoramento de pressão, utilizando ESP32, sensor analógico MPX5700, exibição local em LCD e integração web para monitoramento remoto.

2.2 Objetivos específicos

- Desenvolver um sistema embarcado de baixo custo para leitura e monitoramento contínuo da pressão em tempo real.
- Integrar sensores analógicos, microcontrolador e display local (LCD 1602A).
- Implementar envio de dados via Wi-Fi para uma aplicação web responsiva.
- Estudar a viabilidade de aplicação do ESP32 em tarefas de aquisição de dados, exibição local e transmissão para web.
- Analisar a confiabilidade, clareza de exibição dos dados e potencial de expansão do sistema.

3. Metodologia

A metodologia adotada envolveu pesquisa de componentes, análise de alternativas, montagem em protoboard, programação e testes. O sensor de pressão MPX5700 foi selecionado pela

robustez, precisão e fácil integração com o ESP32. O LCD 1602A foi adotado para exibição local clara, enquanto o ESP32 permitiu aquisição, processamento e transmissão de dados.

Durante o desenvolvimento do projeto, o grupo estabeleceu uma parceria com a empresa **Speed Air**, especializada em soluções para climatização, automação e sistemas de ar-condicionado industrial. Os integrantes realizaram visitas técnicas às instalações da empresa, onde puderam observar, na prática, ambientes industriais equipados com sistemas de monitoramento de pressão, sensores e dispositivos IHM.

Essa aproximação permitiu ao grupo compreender desafios reais na automação de processos industriais, além de colaborar ativamente na montagem e programação de um protótipo de IHM baseado no ESP32, para coleta e exibição de dados de pressão em tempo real. Os profissionais da Speed Air contribuíram com orientações sobre boas práticas de instalação de sensores, segurança elétrica e organização dos componentes.

A experiência prática na Speed Air foi fundamental para consolidar os conhecimentos adquiridos em sala de aula, favorecendo o desenvolvimento de um sistema alinhado com as necessidades do mercado e reforçando a importância da integração entre universidade e indústria para a formação de engenheiros capacitados e atualizados.

A integração com a web foi feita utilizando scripts PHP hospedados em ambiente Replit, para receber, armazenar e disponibilizar os dados em tempo real. O site de monitoramento foi desenvolvido em HTML5, CSS e Javascript com foco em responsividade e visualização clara.

Foram realizados testes empíricos, simulações de variação de pressão, validação de leitura, atualização do display e integração com a web.

4. Desenvolvimento do Sistema

4.1 Arquitetura do sistema elétrico

O sistema utiliza o ESP32 alimentado por fonte 5V (powerbank), com sensor MPX5700 na entrada analógica (GPIO36) e display LCD 1602A em modo 4 bits. Todo o aterramento é comum e a alimentação foi testada para evitar ruídos e garantir estabilidade.

4.2 Programação embarcada (ESP32)

O código principal realiza a leitura do sensor, converte para pressão em kPa, exibe em tempo real no LCD e envia via Wi-Fi para o servidor web.

Listing 1: Código principal do ESP32

```
1 #include <WiFi.h>
2 #include <HTTPClient.h>
3 #include <LiquidCrystal.h>
4
5 // Dados da rede Wi-Fi
```

```

6 const char* ssid = "madeinbrazil";
7 const char* password = "russiachina";
8 const char* serverName = "https://581f5c66-7193-4422-bb5e-
  a65a315ee698-00-2cmlfyuornfw3.picard.replit.dev/index.php";
9
10 // Pinos do LCD (ajuste conforme ligacao)
11 #define LCD_RS 16
12 #define LCD_EN 17
13 #define LCD_D4 18
14 #define LCD_D5 19
15 #define LCD_D6 21
16 #define LCD_D7 22
17 LiquidCrystal lcd(LCD_RS, LCD_EN, LCD_D4, LCD_D5, LCD_D6, LCD_D7);
18
19 const int pinSensor = 36;
20
21 void setup() {
22   Serial.begin(115200);
23   lcd.begin(16, 2);
24
25   lcd.clear();
26   lcd.setCursor(0, 0);
27   lcd.print("Conectando...");
28   lcd.setCursor(0, 1);
29   lcd.print("WiFi_");
30   WiFi.begin(ssid, password);
31   int ponto = 0;
32   while (WiFi.status() != WL_CONNECTED) {
33     delay(500);
34     lcd.setCursor(5, 1);
35     for (int i = 0; i < ponto; i++) lcd.print(".");
36     for (int i = ponto; i < 3; i++) lcd.print("_");
37     ponto = (ponto + 1) % 4;
38   }
39   lcd.clear();
40   lcd.setCursor(0, 0);
41   lcd.print("Conectado!");
42   lcd.setCursor(0, 1);
43   lcd.print(WiFi.localIP());
44   delay(2000);
45 }

```

```

46
47 void loop() {
48     int valor_bruto = analogRead(pinSensor);
49     float tensao = (valor_bruto / 4095.0) * 3.3;
50     float pressao_kPa = (tensao - 0.04) / 0.0012858;
51
52     Serial.print("Pressao_(kPa):_");
53     Serial.println(pressao_kPa);
54
55     bool servidorOK = false;
56     int httpResponseCode = 0;
57     if (WiFi.status() == WL_CONNECTED) {
58         HTTPClient http;
59         String url = String(serverName) + "?pressao=" + String(
            pressao_kPa, 2);
60         http.begin(url);
61         httpResponseCode = http.GET();
62         servidorOK = (httpResponseCode == 200);
63         http.end();
64     } else {
65         Serial.println("WiFi_desconectado!");
66     }
67
68     lcd.setCursor(0, 0);
69     lcd.print("Pressao:");
70     lcd.setCursor(9, 0);
71     if (pressao_kPa < 100.0) lcd.print("_");
72     lcd.print(pressao_kPa, 1);
73     lcd.print("kPa");
74
75     lcd.setCursor(0, 1);
76     if (servidorOK) {
77         lcd.print("Servidor:_OK_");
78     } else {
79         lcd.print("Servidor:_--_");
80     }
81     delay(5000);
82 }

```

4.3 Programação do servidor web (PHP)

O servidor PHP recebe os dados por GET e armazena em um arquivo texto. Outro script retorna todos os dados em JSON.

Listing 2: index.php - Recepção dos dados

```
1 <?php
2 if (isset($_GET['pressao'])) {
3     $pressao = $_GET['pressao'];
4     $data = date('Y-m-d_H:i:s');
5     file_put_contents("dados.txt", "$data;$pressao\n", FILE_APPEND);
6     echo "Salvo:_$data=_$pressao";
7 } else {
8     echo "Use_?pressao=123.4_na_URL_para_enviar_dados!";
9 }
10 ?>
```

Listing 3: dados.php - API para o site

```
1 <?php
2 if (!file_exists("dados.txt")) {
3     echo "[]";
4     exit;
5 }
6 $linhas = file("dados.txt", FILE_IGNORE_NEW_LINES |
7     FILE_SKIP_EMPTY_LINES);
8 $resultado = [];
9 foreach ($linhas as $linha) {
10     list($datahora, $pressao) = explode(";", $linha);
11     $resultado[] = [
12         "timestamp" => $datahora,
13         "pressao" => (float)$pressao
14     ];
15 }
16 header('Content-Type: application/json');
17 echo json_encode($resultado);
18 ?>
```

4.4 Desenvolvimento do site (visualização gráfica responsiva)

O site exibe o gráfico de pressão em tempo real, com visual moderno, responsivo e modo escuro. O valor mais recente é destacado.

Listing 4: grafico.html - Visualização web

```
1 <!DOCTYPE html>
2 <html lang="pt-br">
3 <head>
4   <meta charset="UTF-8">
5   <title>Monitoramento de Pressao</title>
6   <meta name="viewport" content="width=device-width,_initial-scale
7     =1">
8   <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
9   <style>
10     :root {
11       --bg: #111418;
12       --card: #21242b;
13       --text: #f2f2f7;
14       --accent: #3ff063;
15       --shadow: 0 6px 32px #000a;
16       --border-radius: 18px;
17     }
18     html, body {
19       height: 100%;
20       margin: 0;
21       padding: 0;
22       background: var(--bg);
23       color: var(--text);
24       font-family: 'Segoe UI', Arial, sans-serif;
25     }
26     body {
27       min-height: 100vh;
28       display: flex;
29       flex-direction: column;
30       align-items: center;
31       justify-content: flex-start;
32       padding-bottom: 40px;
33     }
34     h2 {
35       margin-top: 32px;
36       font-size: 2.1em;
37       font-weight: 600;
38       letter-spacing: 1px;
39     }
40     #pressaoAtualBox {
```

```

40     display: inline-block;
41     background: var(--card);
42     color: var(--accent);
43     border-radius: var(--border-radius);
44     padding: 20px 46px;
45     font-size: 2.6em;
46     margin: 28px 0 26px 0;
47     box-shadow: var(--shadow);
48     border: 2.5px solid var(--accent);
49     letter-spacing: 1px;
50 }
51 .grafico-container {
52     background: var(--card);
53     border-radius: var(--border-radius);
54     box-shadow: var(--shadow);
55     width: 98vw;
56     max-width: 1200px;
57     margin: 0 auto 0 auto;
58     padding: 40px 10px 25px 10px;
59     display: flex;
60     flex-direction: column;
61     align-items: center;
62 }
63 #pressaoChart {
64     width: 100% !important;
65     height: 600px !important;
66     min-height: 340px;
67     max-height: 600px;
68     background: var(--card);
69     border-radius: var(--border-radius);
70 }
71 @media (max-width: 1300px) {
72     .grafico-container { max-width: 99vw; }
73     #pressaoChart { height: 47vw !important; min-height: 220px; }
74 }
75 @media (max-width: 850px) {
76     #pressaoAtualBox { font-size: 1.3em; padding: 10px 16px; }
77     .grafico-container { padding: 10px 2vw; }
78     #pressaoChart { height: 38vw !important; min-height: 180px; }
79 }
80 footer {

```

```

81     margin-top: 45px;
82     font-size: 0.93em;
83     color: #a0a0a0;
84     opacity: 0.66;
85 }
86 </style>
87 </head>
88 <body>
89     <h2>Pressao em tempo real</h2>
90     <div id="pressaoAtualBox">
91         <span id="pressaoAtual">---</span>
92         <span style="font-size:0.5em;">kPa</span>
93     </div>
94     <div class="grafico-container">
95         <canvas id="pressaoChart"></canvas>
96     </div>
97     <footer>
98         Grupo 11 IHM &bull; 2024 &mdash; Monitoramento Online
99     </footer>
100
101     <script>
102         // URL da sua API de dados:
103         const apiUrl = "https://581f5c66-7193-4422-bb5e-a65a315ee698
104             -00-2cmlfyuornfw3.picard.replit.dev/dados.php";
105
106         // Chart.js
107         const ctx = document.getElementById('pressaoChart').
108             getContext('2d');
109         const chart = new Chart(ctx, {
110             type: 'line',
111             data: {
112                 labels: [],
113                 datasets: [{
114                     label: 'Pressao (kPa)',
115                     data: [],
116                     borderColor: '#40e9c9',
117                     backgroundColor: 'rgba(60,255,230,0.10)',
118                     tension: 0.13,
119                     pointRadius: 4,
120                     pointHoverRadius: 7,
121                     pointBackgroundColor: '#3ff063',

```

```

120         pointBorderColor: '#182'
121     }]
122 },
123 options: {
124     plugins: {
125         legend: {
126             labels: { color: "#b8ffe2", font: { size: 18,
127                 weight: 'bold' } }
128         },
129         tooltip: {
130             enabled: true,
131             backgroundColor: "#222",
132             titleColor: "#3ff063",
133             bodyColor: "#fff"
134         },
135         responsive: true,
136         maintainAspectRatio: false,
137         scales: {
138             x: {
139                 ticks: { color: "#b8ffe2", font: {size: 15},
140                     maxTicksLimit: 10, autoSkip: true },
141                 grid: { color: "#23282c", borderDash: [6, 4]
142                     }
143             },
144             y: {
145                 beginAtZero: false,
146                 ticks: { color: "#fff", font: {size: 16}},
147                 grid: { color: "#23282c", borderDash: [4, 4]
148                     }
149             }
150         }
151     }
152 });
153
154 function atualizarGrafico() {
155     fetch(apiUrl)
156         .then(resp => resp.json())
157         .then(data => {
158             chart.data.labels = data.map(d => d.timestamp);
159             chart.data.datasets[0].data = data.map(d => d.

```

```

        pressao);
157     chart.update();
158     // Atualiza pressao atual
159     if (data.length > 0) {
160         let atual = data[data.length - 1].pressao;
161         document.getElementById('pressaoAtual').
            innerText = atual.toFixed(2);
162     } else {
163         document.getElementById('pressaoAtual').
            innerText = "---";
164     }
165 })
166 .catch(() => {
167     document.getElementById('pressaoAtual').innerText
        = "---";
168 });
169 }
170 atualizarGrafico();
171 setInterval(atualizarGrafico, 2000);
172 </script>
173 </body>
174 </html>

```

4.5 Orçamento dos componentes

Tabela 1: Orçamento dos componentes eletrônicos

Componente	Custo (R\$)
Sensor MPX5010dp	245,00
Display LCD 1602A	27,00
Esp 32	31,36
Fonte	110,00
Protoboard	15,00
Jumpers	17,00
Subtotal	445,36,00

4.6 Testes de funcionamento e resultados

Os testes validaram a estabilidade elétrica, atualização visual contínua dos valores no display e a integração com a plataforma web, permitindo monitoramento remoto em tempo real. O sistema se mostrou robusto para uso acadêmico e como base para futuras expansões industriais.

5. Discussões e Limitações

Entre os desafios enfrentados destacam-se:

- Pequenas oscilações devido a ruído elétrico e variações de temperatura;
- Limitações de desempenho visual do LCD para atualizações muito rápidas;
- Ausência inicial de registro/armazenamento de dados e conectividade, superada com a integração web.

Fica como sugestão futura a implementação de armazenamento local, alarmes e expansão para múltiplos sensores.

6. Conclusão

O projeto atingiu o objetivo de criar um sistema embarcado funcional para monitoramento de pressão, integrando leitura analógica, exibição local e visualização web em tempo real. O uso do ESP32, sensor MPX5700 e LCD 1602A resultou em uma solução robusta, de baixo custo e fácil replicação. A integração com a web amplia as possibilidades de aplicação em ambientes industriais, laboratoriais ou acadêmicos.

7. Referências

- ADAFRUIT. ST7735 TFT Display Library. Disponível em: <https://learn.adafruit.com/1-8-tft-display>. Acesso em: 5 jun. 2025.
- BRASIL. ABNT NBR IEC 60601-1: Equipamentos eletromédicos – Requisitos gerais para segurança básica e desempenho essencial. Associação Brasileira de Normas Técnicas, 2023.
- NXP. MPX5010/5700 Series Integrated Silicon Pressure Sensor. Datasheet oficial. Disponível em: <https://www.nxp.com/docs/en/data-sheet/MPX5700.pdf>. Acesso em: 5 jun. 2025.
- ESPRESSIF. ESP32 Technical Reference Manual. Disponível em: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf. Acesso em: 5 jun. 2025.
- SPEED AIR. Site institucional. Disponível em: <https://www.speedair.com.br>. Acesso em: 5 jun. 2025.
- SENAC. Modelo para o artigo. Disponível em: <https://www1.sp.senac.br/hotsites/blogs/revistainiciacao/>. Acesso em: 7 nov. 2024.

- USP. PTC 3570 – Aula: Segurança em dispositivos médicos e normas IEC. Disponível em: <https://citrus.uspnet.usp.br>. Acesso em: 5 jun. 2025.